

Anomaly Detection Using Autoencoders

Course: SEP 740 – Deep Learning

Instructor: Prof. Anwar Mirza

Group 3:

Ann Hoang

Hongyan Chang

Manisurya Udhisi

Drashtiben Vaghasiya

Ziqi Guo

August 2, 2026

Contents

1	Introduction (Drashtiben Vaghasiya)	3
2	Problem Statement and Background (Drashtiben Vaghasiya)	3
3	Theory and Dataset	4
3.1	Autoencoder-Based Anomaly Detection (Drashtiben Vaghasiya)	4
3.2	KDD Cup 1999 Dataset (Drashtiben Vaghasiya)	5
3.3	Data Preprocessing (Ziqi)	6
3.3.1	Using the Saved Preprocessing Artifacts	9
4	Implementation Details	10
4.1	Basic Autoencoder Model (Ziqi)	10
4.2	Sparse Autoencoder Model (Hongyan)	11
4.3	Deep Autoencoder Model (Anh)	12
5	Source Code Explanation	14
5.1	Data Preprocess Source Code Explanation (Ziqi)	14
5.2	Basic Autoencoder Model Source Code Explanation (Ziqi)	15
5.2.1	Hyperparameter Selection	15
5.2.2	Final Basic Autoencoder Training	16
5.2.3	Threshold Calibration, Evaluation, and Visualization	17
5.3	Sparse Autoencoder Model Source Code Explanation (Hongyan)	17
5.4	Deep Autoencoder Model Source Code Explanation (Anh)	19
6	Results and Discussion	20
6.1	Results of Basic Autoencoder (Ziqi)	20
6.2	Results of Sparse Autoencoder (Hongyan)	24
6.2.1	Results of Sparse Autoencoder	24
6.2.2	Hyperparameter Study	27
6.3	Results of Deep Autoencoder (Anh)	30
6.4	Model Comparison (Manisurya & Anh)	34

6.4.1 Performance Comparison	34
7 Recommendations for Future Work (Ziqi & Anh)	36

1 Introduction (Drashtiben Vaghasiya)

Modern enterprise network infrastructures generate massive volumes of high speed data traffic, creating an increasingly complex environment for cybersecurity management. Automated Network Intrusion Detection Systems (NIDS) serve as critical defensive mechanisms designed to continuously monitor network traffic for unauthorized access, malicious policy breaches, and denial of service attempts. NIDS frameworks generally fall into two main paradigms: signature-based detection and anomaly-based detection.

Signature-based systems rely on static, pre-compiled databases of known attack fingerprints. While highly precise for recognized threats, signature matching fails when encountering novel zero-day exploits, modified attack strains, or obfuscated payloads. Conversely, anomaly-based NIDS establishes a statistical or machine-learned baseline of benign (normal) traffic behavior. Any significant deviation from this learned operational baseline is flagged as a potential security anomaly [1, 2].

Unsupervised deep learning—specifically autoencoder neural networks—offers a compelling solution for anomaly detection [1]. Autoencoders map input features into a low-dimensional bottleneck latent space before reconstructing the original input. When trained exclusively on legitimate network traffic, the model captures the structural and statistical correlations of normal operational behavior [1]. Consequently, when presented with malicious traffic during inference, the autoencoder produces significantly higher reconstruction errors. This report presents a comparative evaluation of three autoencoder variants: Basic, Sparse, and Deep Autoencoders, evaluated on the benchmark KDD Cup 1999 network intrusion dataset [3]. In addition to assessing overall detection performance, the study looks at how network depth and sparsity hyperparameters affect anomaly detection, providing valuable insights into the trade-offs between model complexity and detection capabilities for different attack types.

2 Problem Statement and Background (Drashtiben Vaghasiya)

Supervised classification models for network intrusion detection encounter two primary operational bottlenecks in real world deployment:

1. **Severe Class Imbalance:** In production networks, legitimate traffic overwhelmingly outnumbers malicious connections. Acquiring balanced, comprehensive training datasets containing labeled examples for every potential attack vector is practically impossible.
2. **Inability to Detect Zero-Day Exploits:** Supervised models are strictly bounded by their training labels and struggle to classify attack signatures that were absent during initial training.

Unsupervised autoencoders overcome these limitations by framing intrusion detection as a reconstructive anomaly scoring problem [4]. By training strictly on normal connection records, the network learns to reconstruct benign traffic with high fidelity (resulting in a small Mean Squared Error). When malicious traffic passes through the network during inference, its unfamiliar structural characteristics lead to poor reconstruction (yielding a significantly higher Mean Squared Error).

By calibrating an anomaly threshold τ on the distribution of normal reconstruction errors, the model can differentiate benign traffic from malicious intrusions without requiring labeled attack samples during training. The primary goal of this research is to conduct a systematic comparison of Basic, Sparse, and Deep Autoencoders for network intrusion detection using a unified experimental framework. Using the same preprocessing pipeline, hyperparameter tuning technique, and assessment methodology, the study looks at how network depth, sparsity regularization, and threshold calibration affect anomaly detection performance across different attack categories.

3 Theory and Dataset

3.1 Autoencoder-Based Anomaly Detection (Drashtiben Vaghasiya)

An autoencoder is a feedforward neural network designed to reconstruct its input $x \in \mathbb{R}^d$ through a compressed latent bottleneck code $z \in \mathbb{R}^k$ (where $k < d$) [1]. The network consists of two functional transformations:

- **Encoder (f_θ):** Compresses the high dimensional input x into a latent code
- $$z = f_\theta(x) = \sigma(W_e x + b_e).$$

- **Decoder (g_ϕ):** Reconstructs the original input from the latent code

$$\hat{x} = g_\phi(z) = \psi(W_d z + b_d).$$

Network parameters $\Theta = \{\theta, \phi\}$ are optimized over N normal training records by minimizing the Mean Squared Error (MSE) reconstruction loss:

$$\mathcal{L}_{\text{MSE}}(x, \hat{x}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2$$

During evaluation, an anomaly threshold τ is nonparametrically calibrated using high percentiles (e.g., 95th percentile) of the normal calibration reconstruction errors. Any test sample with a reconstruction error exceeding τ is flagged as an anomaly.

3.2 KDD Cup 1999 Dataset (Drashtiben Vaghasiya)

The experiments in this project utilize the benchmark KDD Cup 1999 dataset [3]. The dataset was constructed by processing raw DARPA 98 TCP/IP dump files collected from a simulated military network environment, capturing a wide variety of legitimate sessions alongside synthetic cyberattacks.

The official 10% subset training file (`kddcup.data_10_percent_corrected`) contains **494,021** connection records [3]. Each connection record is described by **41** input attributes (38 numerical variables and 3 categorical variables) along with a target label designating the session as either `normal` or a specific attack type.

The 41 input attributes are grouped into four structural categories:

1. **Basic Connection Features (Attributes 1-9):** General connection metrics extracted directly from TCP/IP headers, including `duration`, `protocol_type` (TCP, UDP, ICMP), `service` (e.g., HTTP, FTP, SMTP, Private), `flag` (connection status like SF, REJ, S0), `src_bytes`, and `dst_bytes`.
2. **Content Features (Attributes 10-22):** Domain-specific features evaluated within the connection payload to detect root/user privilege escalations or system file access, including `logged_in`, `num_compromised`, `root_shell`, `su_attempted`, and `is_guest_login`.

3. **Time-Based Traffic Features (Attributes 23-31):** Aggregate statistics computed over a 2-second time window, such as `count` (number of connections to the same host) and `srv_count` (number of connections to the same service).
4. **Host-Based Traffic Features (Attributes 32-41):** Historical window metrics evaluated across the last 100 connections to assess destination host patterns, such as `dst_host_count`, `dst_host_srv_count`, and `dst_host_same_src_port_rate`.

Malicious records in the dataset are categorized into **four major attack families**:

- **Denial of Service (DoS):** Attacks aimed at overwhelming target computing resources or network bandwidth, rendering services unavailable to legitimate users. Examples include `neptune` (SYN flood), `smurf` (ICMP echo flood), `back`, `teardrop`, `pod`, and `land`.
- **Surveillance / Probe:** Reconnaissance scanning activities executed to map active network hosts, open ports, and vulnerable services. Examples include `satan`, `ipsweep`, `nmap`, and `portsweep`.
- **Remote to Local (R2L):** Unauthorized access attempts initiated by a remote machine attempting to gain local user privileges on a victim host. Examples include `guess_passwd`, `warezmaster`, `warezclient`, `ftp_write`, `imap`, and `multihop`.
- **User to Root (U2R):** Local privilege escalation attacks where an adversary with local non-privileged user access exploits system vulnerabilities to gain superuser (root) privileges. Examples include `buffer_overflow`, `loadmodule`, `perl`, and `rootkit`.

Table 1 summarizes the major attack categories present in the benchmark dataset prior to data cleaning and deduplication.

3.3 Data Preprocessing (Ziqi)

The preprocessing workflow used the KDD Cup 1999 dataset [3]¹ as the data source.

Firstly, the official KDD Cup 1999 dataset file `kddcup.names.txt` was read to restore the 41 input feature names and the traffic label column. The original training file contains 494,021

¹The original KDD Cup 1999 dataset website was not accessible, so this project used the same KDD Cup 1999 dataset from Kaggle: <https://www.kaggle.com/datasets/galaxyh/kdd-cup-1999-data>.

Table 1: Summary of Attack Families in the KDD Cup 1999 Benchmark.

Category	Nature of Threat	Primary Attack Types Included
Normal	Benign Network Sessions	<code>normal</code>
DoS	Resource/Bandwidth Flooding	<code>neptune</code> , <code>smurf</code> , <code>back</code> , <code>teardrop</code> , <code>pod</code> , <code>land</code>
Probe	Reconnaissance / Port Scans	<code>satan</code> , <code>ipsweep</code> , <code>nmap</code> , <code>portsweep</code>
R2L	Remote Password/Access Compromise	<code>guess_passwd</code> , <code>warezmaster</code> , <code>warezclient</code> , <code>ftp_write</code> , <code>imap</code>
U2R	Local Privilege Escalation	<code>buffer_overflow</code> , <code>loadmodule</code> , <code>perl</code> , <code>rootkit</code>

records, with each row consisting of the expected 42 columns, including 41 features and one label. Basic data quality checks revealed that there were no missing values, empty string fields, or non-finite numerical values in the training data. The original labels were initially ended with a period, such as `normal.` and `neptune.`, so the labels were cleaned and standardized before being mapped to binary normal/abnormal labels and attack families. The attack family mapping classified attack records into four categories: DoS, Probe, R2L, and U2R.

During preprocessing, a large number of duplicate records were identified in the KDD Cup 1999 dataset [3]. Therefore, completely duplicate records were removed before model development. By using all 41 feature columns and the traffic labels for duplicate detection, 348,435 redundant data rows were removed from the original 494,021 training records. Additionally, the preprocessing step also checked for ambiguous records with the same feature values but different labels. There were label conflicts in two feature groups, resulting in 4 conflicting records, including one record with the same feature values being labeled as normal traffic and attack. These ambiguous records were deleted to avoid providing inconsistent training or evaluation signals to the model. After removing the duplicate and conflicting records, the cleaned development dataset contained 145,582 unique records. The cleaned data was then classified based on its role in the anomaly detection workflow. Since the autoencoder is designed to learn the normal traffic patterns, only normal records were used for model training. The cleaned normal dataset contained 87,831 records, and the cleaned abnormal reference dataset contained 57,751 records. The normal dataset was randomly divided into a seed 42, with 70% for training data, 15% for validation data, and 15% for threshold calibration data. This The segmentation dimensions of the result are as follows.

The feature transformation step was designed to reduce feature-scale imbalance while avoid-

Data split	Number of records
Normal training	61,481
Normal validation	13,175
Normal calibration	13,175
Anomaly	57,751

Table 2: Preprocessed KDD Cup 1999 data splits used in this project.

ing data leakage. A `ColumnTransformer` was fitted only on the normal training split, meaning that all scaling parameters and categorical encoding rules were learned only from normal training records. The validation, calibration, anomaly reference, and future final test data were transformed using this fitted transformer only. For nine highly skewed non-negative numeric features, `log1p` was before Min-Max scaling. This was necessary because these features contain very large values and strongly right-skewed distributions. Without the logarithmic transformation, a few extreme values could dominate the scaling range and compress most normal records into a very small interval. The `log1p` transformation reduces the effect of extreme values while still preserving zero values. After this step, Min-Max scaling mapped the transformed numeric values into the range $[0, 1]$. The remaining numeric features were scaled directly with Min-Max scaling, and the three categorical features, `protocol_type`, `service`, and `flag`, were encoded with one-hot encoding using `handle_unknown='ignore'` so that unseen categories in later data could be handled safely.

All 41 original input features were retained, rather than removing the constant or nearly constant features that were present in the normal training set. This decision is crucial for anomaly detection because the autoencoder only learns normal traffic patterns. If a certain feature is always zero in normal traffic but becomes non-zero in attack records, this change could potentially serve as a useful anomaly signal.

After transformation, each sample was represented as a 74-dimensional `float32` vector. All processed feature matrices contained only finite values and were scaled to the range $[0, 1]$. The final preprocessing outputs were saved under the `artifacts` folder: `kdd99_preprocessed_data.npz` stores the processed arrays, labels, attack categories, and original row indices; `kdd99_preprocessor.joblib` stores the fitted preprocessing pipeline; and `kdd99_preprocessing_metadata.json` records the feature schema, transformation rules, cleaning counts, split sizes, and intended role of each dataset. These saved artifacts allow the later autoencoder training, threshold calibration, valu-

ation, and visualization scripts to reuse the same preprocessed data without rerunning the full notebook.

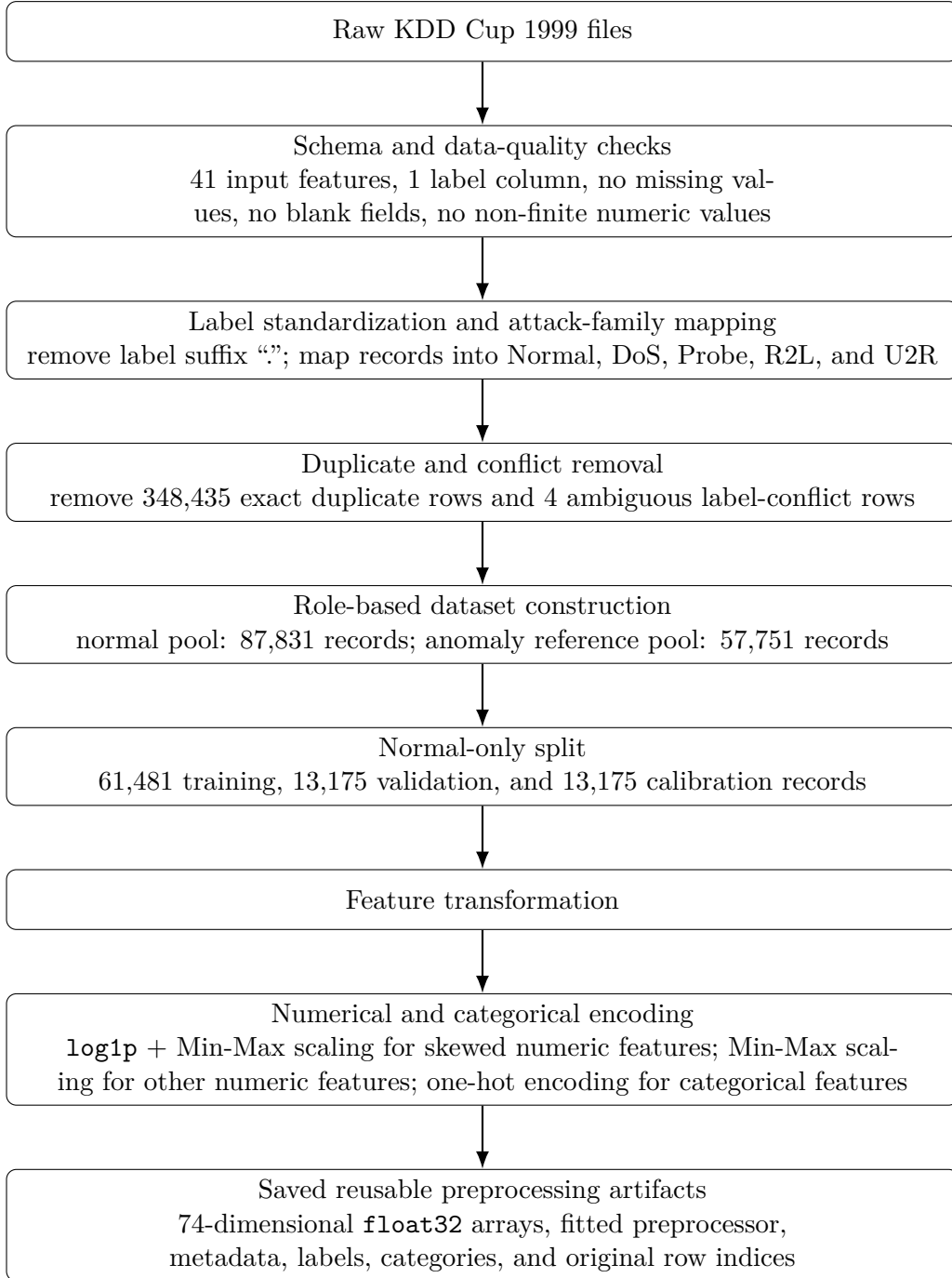


Figure 3.1: Preprocessing workflow.

3.3.1 Using the Saved Preprocessing Artifacts

After preprocessing, other team members can reuse the saved artifacts directly. The main training and evaluation arrays can be loaded as follows:

Listing 1: Loading the preprocessed KDD Cup 1999 arrays.

```
import numpy as np

data = np.load( "artifacts/kdd99_preprocessed_data.npz", allow_pickle=False,)

X_train_normal = data["X_train_normal"] # train autoencoder
X_validation_normal = data["X_validation_normal"] # early stopping / model selection
X_calibration_normal = data["X_calibration_normal"] # threshold calibration
X_anomaly_reference = data["X_anomaly_reference"] # evaluation
y_anomaly_reference = data["y_anomaly_reference"] # evaluation
```

4 Implementation Details

4.1 Basic Autoencoder Model (Ziqi)

The basic autoencoder was implemented as a fully connected neural network using PyTorch [5]. Since the preprocessed KDD Cup 1999 [3] data contains 74 numerical input features, each traffic record is represented as a vector $x \in \mathbb{R}^{74}$. The purpose of the autoencoder is to learn a compressed representation of normal network traffic and then reconstruct the original input from this representation.

The model contains two main parts: the encoder and decoder. The encoder reduces the input dimension from 74 to a 16-dimensional latent space, while the decoder expands this latent representation back to the original 74-dimensional feature space. The final architecture is:

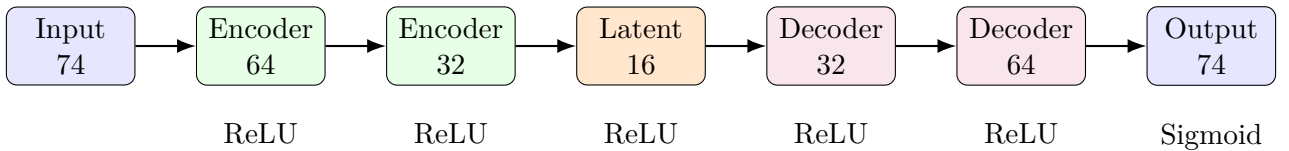


Figure 4.1: Structure of the basic autoencoder model.

ReLU activation functions are used after the hidden layers because they are simple, efficient, and help the network learn nonlinear relationships in the data. The output layer uses a sigmoid activation function because all input features were scaled to the range $[0, 1]$ during

preprocessing. This makes the reconstructed output consistent with the scale of the input data.

The autoencoder is trained only using normal traffic records. During training, the input itself is used as the reconstruction target. The model therefore learns to minimize the difference between the original input x and the reconstructed output $\hat{x}$. Mean squared error is used as the reconstruction loss:

$$\mathcal{L}(x, \hat{x}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2,$$

where $d = 74$ is the number of input features. After training, this reconstruction error is also used as the anomaly score. Normal records are expected to have relatively small reconstruction errors, while attack records are expected to produce larger reconstruction errors because they do not follow the normal traffic patterns learned by the model.

4.2 Sparse Autoencoder Model (Hongyan)

The sparse autoencoder is implemented as an advanced model that extends the basic autoencoder with a sparsity constraint on the hidden representation. The motivation is that, instead of only compressing the input into a dense latent code, the model is encouraged to keep most hidden units inactive for any given input. This is intended to encourage the encoder to learn a small set of specialized and more meaningful features of normal traffic, with the aim of improving the separation between normal and attack records in the reconstruction-error space, particularly for the rare attack categories [6].

Like the basic autoencoder, the sparse autoencoder is a fully connected network that maps a 74-dimensional input vector $x \in \mathbb{R}^{74}$ to a reconstruction $\hat{x}$, and it is trained only on normal traffic. To allow a fair comparison, it uses the same layer sizes as the final basic autoencoder, namely a symmetric encoder-decoder design $74 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 74$, as shown in Figure 4.2. The main difference from the basic model is the choice of activation function in the encoder: every encoder layer uses a sigmoid activation instead of ReLU. This is a deliberate design choice, because the sparsity penalty interprets each hidden activation as an activation probability, which must lie in the range $[0, 1]$. The decoder keeps ReLU activations for its hidden layers and a sigmoid activation at the output layer so that the reconstructed values

match the $[0, 1]$ range of the preprocessed input.

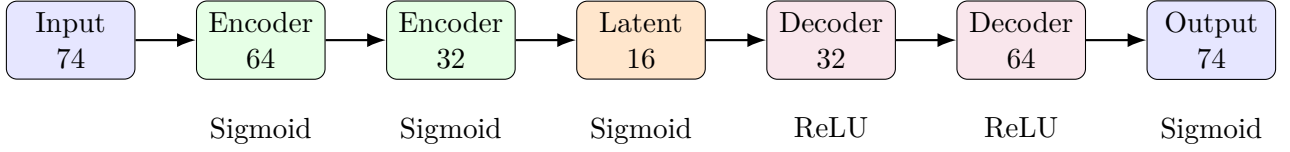


Figure 4.2: Structure of the sparse autoencoder model.

The training objective combines the reconstruction loss with a sparsity penalty. As in the basic model, the reconstruction quality is measured by the mean squared error between the input x and the reconstruction $\hat{x}$. For the sparsity term, let $\hat{\rho}_j$ denote the average activation of encoder unit j over a mini-batch, and let ρ be a small target activation, called the sparsity target. The deviation of $\hat{\rho}_j$ from ρ is penalized using the Kullback–Leibler (KL) divergence between two Bernoulli distributions [6]:

$$\text{KL}(\rho \parallel \hat{\rho}_j) = \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}.$$

The total loss is the mean squared reconstruction error plus the sparsity penalty summed over all encoder units, weighted by a sparsity weight β [6]:

$$\mathcal{L}(x, \hat{x}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2 + \beta \sum_j \text{KL}(\rho \parallel \hat{\rho}_j),$$

where $d = 74$ is the number of input features. The hyperparameter ρ controls how sparse the hidden representation should be, with a smaller ρ enforcing stronger sparsity, while β controls how strongly the sparsity constraint is applied relative to reconstruction. Both hyperparameters are studied experimentally in Section 6.2.2. After training, the anomaly score is still the mean squared reconstruction error, and the same percentile-based threshold calibration procedure as the basic model is used, so that the two models remain directly comparable.

4.3 Deep Autoencoder Model (Anh)

Another specialized model is the deep autoencoder, which expands the network by adding more hidden layers to investigate how depth affects reconstruction quality and anomaly detection performance. The hypothesis is that additional depth allows the model to capture more com-

plex non-linear relationships and information in normal traffic. Unlike the basic and sparse autoencoder models, which were implemented in PyTorch, the deep autoencoder was implemented using TensorFlow [7]. TensorFlow was chosen to gain experience with an alternative deep learning framework and because its high-level Keras API provides built-in training utilities that simplify model implementation. The framework was selected for implementation convenience rather than an expectation of improved model performance.

The hyperparameters, such as learning rate and latent space, were inherited from the basic model’s best configuration. However, as Figure 4.3 illustrates, two further hidden layers were added to either side of the network. As a result, rather than going through two hidden layers as in a basic or sparse model, the model goes through four on each encode and decode side. As in the basic model, ReLU activations are employed across the hidden layers for their simplicity and ability to simulate nonlinear relationships, while the output layer uses a sigmoid activation to match the $[0, 1]$ scaling of the preprocessed input.

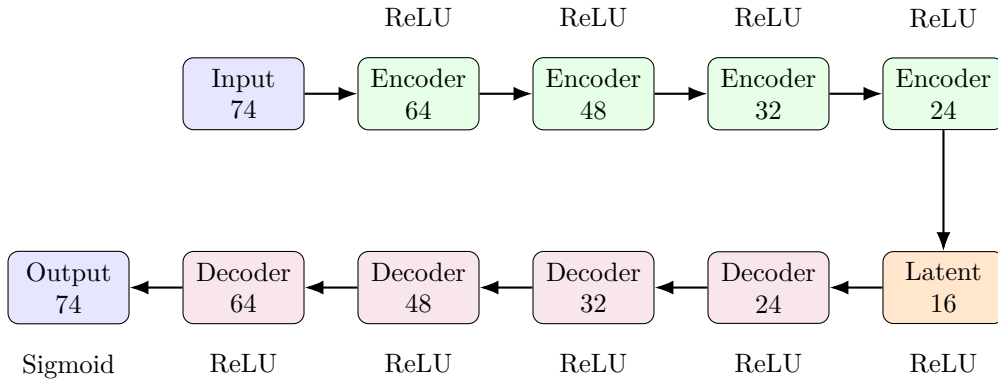


Figure 4.3: Structure of the deep autoencoder model.

The mean squared error, like the basic model, is a statistic used to evaluate the reconstruction performance with the Adam optimizer [8]. This ensures that the training setting remains consistent with the fundamental model, and any variations in performance can be ascribed to the change in architecture rather than the optimization technique. After training, all three models use a percentile-based threshold calibration technique to calculate the anomaly detection threshold, ensuring consistency in evaluation. Calibration is performed using only normal traffic to determine the threshold percentile values, after which the trained model is evaluated on unseen attack traffic to measure how well it distinguishes anomalies from normal behavior.

5 Source Code Explanation

The complete implementation of this project, including the preprocessing, model training, evaluation, and visualization scripts, is available in the GitHub repository: <https://github.com/hongyanchang/SEP740-DeepLearning.git>

5.1 Data Preprocess Source Code Explanation (Ziqi)

The data preprocessing code is implemented in the `data_preprocessing.ipynb` file. This code converts the original KDD Cup 1999 [3] traffic records into clean, numerical arrays suitable for training the autoencoder. The preprocessing process also clarifies the roles of different data sets so that the autoencoder to learn only from normal traffic.

First, the code loads the official KDD Cup 1999 [3] feature definition file and creates column names for the original dataset. Then, the training file `kddcup.data_10_percent_corrected` is read into a Pandas data frame. This notebook checks the row length, missing values, empty strings, non-finite numbers, and label format. Since the KDD labels in the original file end with a period, the code removes the period and creates a binary anomaly indicator, where normal traffic is assigned a value of 0 and attack traffic is assigned a value of 1.

The next part of the code analyzes the labels and attack categories in the dataset. The attack names are mapped to the KDD attack categories. Subsequently, redundant and ambiguous records are deleted. By removing duplicate rows, deviations caused by duplicate flow records are reduced. Additionally, records with the same feature values but conflicting labels are also deleted because these records may cause unclear supervisory information during the evaluation process. After cleaning, the dataset is divided into a normal traffic pool and an abnormal reference pool. Only the normal traffic pool is used to construct the training set, validation set, and calibration set.

The normal records are split into three disjoint subsets using a fixed random seed. The training set is used to fit the autoencoder. The validation set is used for model selection and early stopping. The calibration set is reserved for estimating the anomaly threshold after training. The anomaly set is kept separate and is used only for development evaluation.

For feature transformation, the code builds a Scikit-learn `ColumnTransformer` [9]. Three

categorical features, `protocol_type`, `service`, and `flag`, are converted with one-hot encoding. The encoder uses `handle_unknown="ignore"` so that unseen categories in validation, calibration, anomaly, or final test data do not break the transformation. Several highly skewed numeric features, such as `duration`, `src_bytes`, `dst_bytes`, `count`, and `srv_count`, are transformed with `log1p` before Min-Max scaling. The remaining numeric features are scaled directly to the range $[0, 1]$.

A key design choice is that the preprocessor is fitted only on the normal training data. The validation, calibration, anomaly reference, and final test data are transformed using the already fitted preprocessor. This is important because fitting preprocessing parameters on non-training data would leak information from evaluation data into the model development process.

The final output of preprocessing consists of 74-dimensional `float32` feature matrices. The code saves these arrays into `artifacts/kdd99_preprocessed_data.npz`, saves the fitted transformer as `artifacts/kdd99_preprocessor.joblib`, and saves metadata such as feature names, split sizes, transformation rules, and data roles in `artifacts/kdd99_preprocessing_metadata.js`. These saved artifacts allow the later training, threshold calibration, evaluation, and visualization scripts to reuse the same preprocessing pipeline consistently.

5.2 Basic Autoencoder Model Source Code Explanation (Ziqi)

5.2.1 Hyperparameter Selection

The hyperparameter selection process is implemented in `select_basic_autoencoder_config.py`. This script does not train the final submitted model directly. Instead, it trains several temporary candidate autoencoders and compares their performance in order to select a suitable configuration for the final training stage.

The candidate search uses a small grid of model widths, latent dimensions, and learning rates. The hidden-layer options are (48, 24) and (64, 32), the latent dimensions are 8, 12, and 16, and the learning rates are 10^{-3} and 5×10^{-4} . This gives 12 candidate configurations in total. Each candidate uses the same training function, batch size, optimizer type, weight decay, and early stopping logic, so the comparison is consistent.

After each candidate model is trained on `X_train_normal`, the script evaluates it using

reconstruction errors. A temporary threshold is computed from the 95th percentile of the normal calibration reconstruction errors. The candidate is then tested against the calibration normal data and the anomaly reference data. The script records validation loss, threshold value, precision, recall, F1 score, accuracy, and average reconstruction errors.

The best configuration is selected mainly by the highest F1 score under the temporary p95 threshold. If two configurations have similar F1 scores, the one with lower validation reconstruction loss and fewer trained epochs would be chosen. The selected configuration is saved to `artifacts/hyperparameter_search/basic_autoencoder_best_config.json`. In the completed run, the selected configuration was `wider_width_latent16_lr0p001`, corresponding to the architecture

$$74 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 74.$$

5.2.2 Final Basic Autoencoder Training

The final model training code is implemented in `train_final_basic_autoencoder.py`. This script loads the selected hyperparameter configuration and trains the final basic autoencoder from scratch. This separation is important because the models trained during hyperparameter selection are only temporary candidates, while this stage produces the actual saved model used for later threshold calibration, evaluation, and visualization.

The training loop uses mini-batches from `X_train_normal`. After each epoch, the model is evaluated on `X_validation_normal`. Early stopping is controlled by the validation loss. If the validation loss improves by at least `min_delta`, the current model weights are saved as the best weights. If the validation loss does not improve for `patience` epochs, training stops early and the best weights are restored. This prevents the final model from overfitting the normal training data.

After training, the model checkpoint is saved to `artifacts/models/basic_autoencoder.pt`. The checkpoint includes the model weights, input dimension, selected configuration, best epoch, validation loss, number of trained epochs, and random seed. The training and validation losses for each epoch are also saved to `artifacts/training_history/basic_autoencoder_history.csv` for later plotting.

5.2.3 Threshold Calibration, Evaluation, and Visualization

After the final autoencoder is trained, `calibrate_basic_threshold.py` computes anomaly thresholds from the reconstruction errors of `X_calibration_normal`. This calibration set contains only normal traffic and is not used for model training. The script computes several percentile thresholds, including p90, p95, p97.5, and p99. The p95 threshold is used as the primary threshold for the basic model. A record is classified as anomalous if its reconstruction error is greater than the selected threshold.

The evaluation code is implemented in the file `evaluate_basic_autoencoder.py`. This file loads the final trained model, calibrated thresholds, normal calibration data, and abnormal reference data. For each record, the mean square reconstruction error is calculated, and the error is converted into binary prediction results using each threshold. The script then reports precision, recall, F1 score, accuracy, false positive rate, specificity, confusion matrix counts, and recall by attack category. Detailed results are saved in the directory `artifacts/evaluation`.

The visualization code is implemented in the file `visualize_basic_results.py`. This script reads the saved training history, reconstruction errors, thresholds, and evaluation metrics, and then generates charts for reporting. The generated charts include training and validation loss curves, reconstruction error distributions, comparisons of precision/recall/F1 under different thresholds, precision-recall curves, and confusion matrices corresponding to the main thresholds. These charts will be saved in the directory `artifacts/figures/basic_autoencoder`.

Finally, `run_basic_autoencoder_pipeline.py` connects all stages into one ordered workflow: hyperparameter selection, final model training, threshold calibration, evaluation, and visualization which makes the experiment reproducible.

5.3 Sparse Autoencoder Model Source Code Explanation (Hongyan)

The sparse autoencoder follows the same pipeline as the basic model, so that the two models share an identical data flow and evaluation procedure. All scripts write their outputs using a `sparse` prefix to ensure none of the basic autoencoder artifacts are overwritten.

The model definition and training loop are implemented in `train_sparse_autoencoder.py`. The encoder layers are defined individually so that their intermediate activations can be col-

lected during the forward pass, because the sparsity penalty is computed from these activations. The script implements the KL-divergence sparsity penalty described in Section 4.2 and adds it to the mean squared reconstruction error, weighted by the sparsity weight β . The model is trained only on `X_train_normal` using the Adam optimizer [8], and it is evaluated on `X_validation_normal` after each epoch. The model with the lowest validation reconstruction error is saved to `artifacts/models/sparse_autoencoder.pt`. Training history (reconstruction/sparsity) is stored in `artifacts/training_history/sparse_autoencoder_history.csv`. Model selection is based on the validation reconstruction error rather than the total loss, so that the threshold calibration remains comparable with the basic model.

Threshold calibration is implemented in `calibrate_sparse_threshold.py`. It loads the trained model, computes the reconstruction errors on `X_calibration_normal`, and derives the p90, p95, p97.5, and p99 percentile thresholds, using p95 as the primary threshold. The thresholds are saved to `artifacts/thresholds/sparse_threshold.json`.

The evaluation code is implemented in `evaluate_sparse_autoencoder.py`. It computes reconstruction errors on the normal calibration data and the anomaly reference data, applies each calibrated threshold, and reports precision, recall, F1 score, accuracy, false positive rate, specificity, confusion-matrix counts, and recall by attack category. The results are saved under `artifacts/evaluation` as `sparse_metrics.json`, `sparse_metrics.csv`, and `sparse_reconstruction_errors.npz`.

The visualization code is implemented in `visualize_sparse_results.py`. It reads the saved training history, reconstruction errors, thresholds, and evaluation metrics, and produces the training curve, the reconstruction error distribution, the precision/recall/F1 versus threshold plot, the precision-recall curve, and the confusion matrix. All figures are saved under `artifacts/figures/sparse_autoencoder`.

Furthermore, two additional python scripts are implemented for the hyperparameter studies. `experiment_sparse_beta.py` trains the model several times with different sparsity weights β while keeping every other setting fixed, and `experiment_sparse_rho.py` does the same for the sparsity target ρ . Both scripts recalibrate a p95 threshold and evaluate on the anomaly reference set for each configuration, and they save all comparison tables and figures under `artifacts/experiments`. These experiments are kept separate from the main pipeline so that

the primary sparse autoencoder artifacts are never modified.

5.4 Deep Autoencoder Model Source Code Explanation (Anh)

In contrast to the simple autoencoder, the deep model employed fixed hyperparameters, as conducted in Section 5.2.1. For easy comparison, the trained model and its evaluation outputs are kept in the artifacts folder with a `deep` prefix.

The model is implemented in `train_final_deep_autoencoder.py`. The network’s hidden, latent, and output layers are constructed using TensorFlow’s `Dense` layers. The Adam optimizer [8] is employed with a learning rate of 0.001, a weight decay of 1e-5, and `mse` as the loss function, as stated in Section 4.3. With a patience of 8 and a `min_delta` of 1e-6, early stopping is employed to avoid overfitting, as inherited from the optimal configuration of the basic model documented in `basic_autoencoder_best_config.json`. The model with the lowest validation loss is saved to `artifacts/models/deep_autoencoder.weights.h5`, and its architecture at that point is saved to `artifacts/models/deep_autoencoder.weights.json`. The training and validation loss history is saved in `artifacts/training_history/deep_autoencoder_history.csv`.

The implementation of threshold calibration is found in `calibrate_deep_threshold.py`. Using the input size stored in `artifacts/models/deep_autoencoder.weights.json`, it constructs the deep autoencoder model and loads the optimal model’s training parameters to calculate its reconstruction error on each record in `X_calibration_normal`. Four candidate thresholds at the p90, p95, p97.5, and p99 percentiles are obtained from this distribution of normal errors. Higher percentiles decrease false positives at the expense of overlooking subtle assaults, whereas lower percentiles detect more abnormalities but generate more false positives. P95 is chosen to be the major threshold, and all four values are stored in `artifacts/thresholds/deep_threshold.json`.

The reconstruction errors on both the normal and anomalous reference data are calculated by the evaluation code in `evaluate_deep_autoencoder.py`. It measures detection performance by applying the calibrated threshold. Precision, recall, F1 score, accuracy, false positive rate, specificity, confusion-matrix counts, and recall by attack type are all reported. The findings are displayed in `artifacts/evaluation`.

The visualization code from the basic model is utilized to load preserved training history,

reconstruction errors, thresholds, and evaluation metrics, generating a final comparison for the deep model. All figures are stored as `artifacts/figures/deep_autoencoder`.

6 Results and Discussion

6.1 Results of Basic Autoencoder (Ziqi)

The basic autoencoder results were generated on a Windows 11 machine with an Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz. The experiments were executed using Python 3.12.1, PyTorch 2.12.1+cpu [5], NumPy 2.4.6 [10], and scikit-learn 1.9.0 [9]. All model training and evaluation were performed on CPU. PyTorch [5] used 4 CPU threads and 4 inter-operation threads during the experiment. A fixed random seed of 42 was set for Python, NumPy, and PyTorch [5] during model training. Under this environment, re-running the complete basic autoencoder pipeline reproduced the same selected configuration and evaluation metrics. However, minor numerical differences may occur when the same code is re-run on other machines or operating systems because neural-network training depends on low-level floating-point operations, PyTorch [5] build details, CPU backends, processor architecture, and thread scheduling. These small differences can accumulate over training epochs and may slightly change validation losses, reconstruction-error thresholds, F1 scores, and in rare cases the selected hyperparameter configuration. Therefore, the results reported in this project are based on the saved experimental artifacts generated in the stated environment.

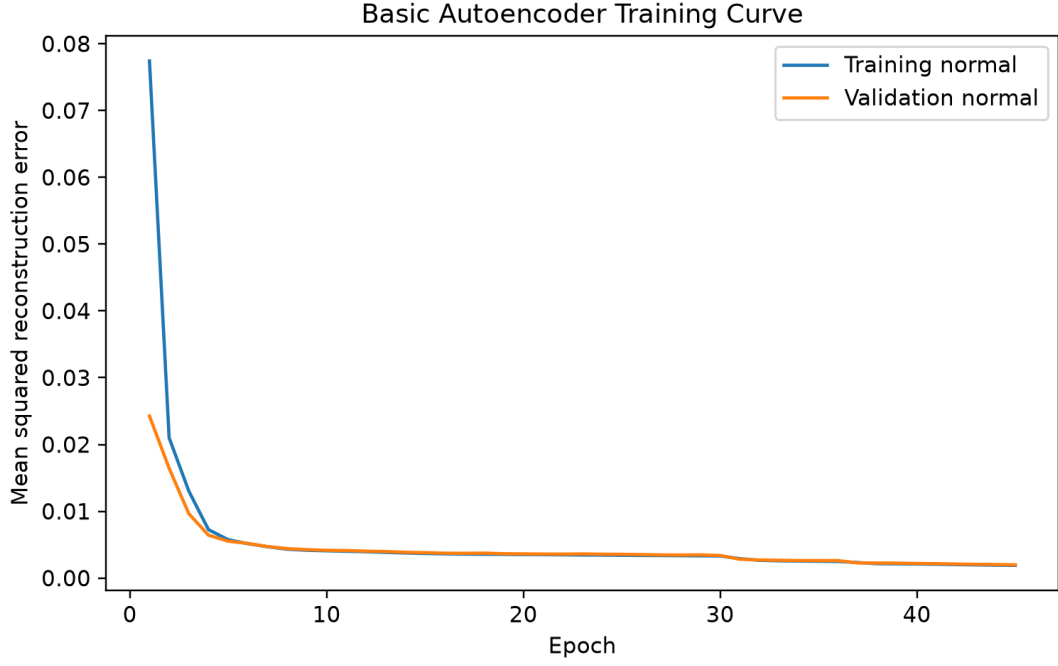


Figure 6.1: Training and validation reconstruction loss of the basic autoencoder.

Figure 6.1 presents the training and validation loss curves. In the initial few cycles, both losses decreased rapidly and then stabilized. The validation loss was closely consistent with the training loss, indicating that the model learned the normal traffic patterns without significant overfitting.

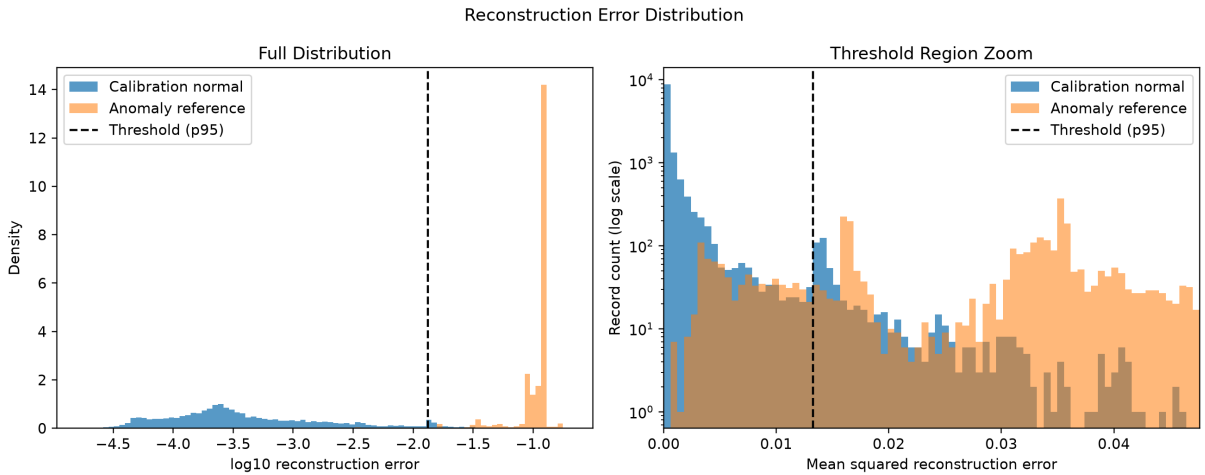


Figure 6.2: Reconstruction error distribution.

Figure 6.2 compares the distribution of reconstruction errors for normal and abnormal records. Normal calibration records are concentrated around very small reconstruction errors, while the errors of most abnormal records are significantly larger. The average reconstruction

error of normal calibration records is 0.002, while that of abnormal records is 0.11. This obvious difference supports using reconstruction errors as an abnormality scoring metric. However, there is still some overlap in the enlarged threshold area, which explains why there are still false positives and false negatives.

Table 3: Basic autoencoder performance under different percentile thresholds.

Threshold	Value	Precision	Recall	F1	Accuracy
p90	0.0041	0.9776	0.9966	0.9870	0.9786
p95	0.0133	0.9886	0.9866	0.9876	0.9798
p97.5	0.0158	0.9942	0.9846	0.9894	0.9828
p99	0.0257	0.9977	0.9739	0.9856	0.9769

Table 3 presents the trade-off relationship among different thresholds. Lower thresholds (such as p90) can detect more anomalies and thus achieve the highest recall rate, but they also generate more false positives. Higher thresholds (such as p99) can reduce the false positive rate, but they will miss more attacks. In this evaluation, the F1 score of p97.5 has the best F1-score.

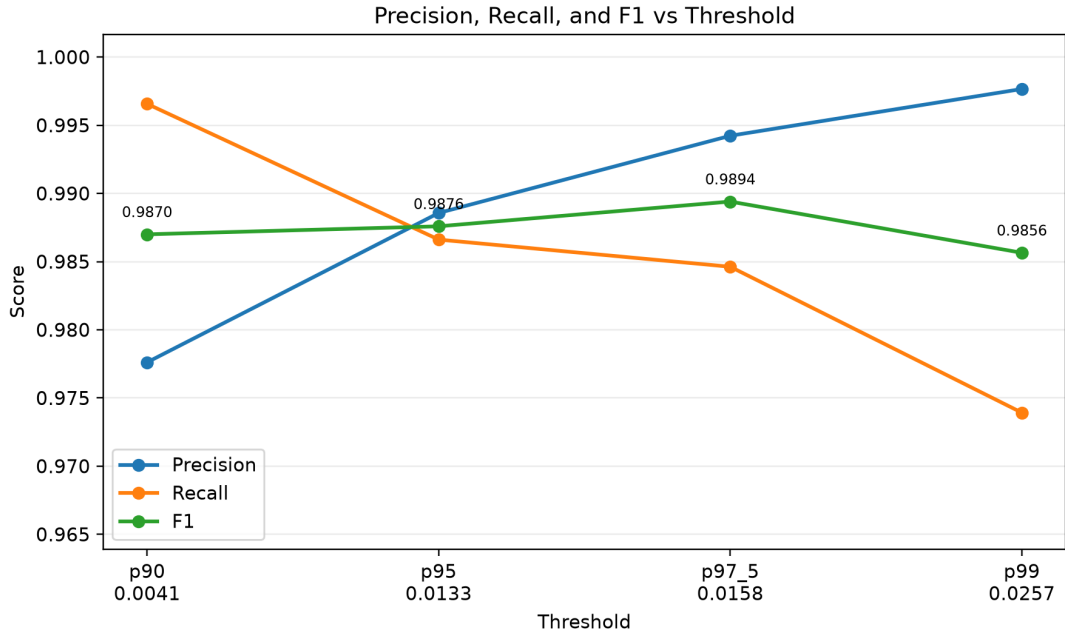


Figure 6.3: Precision, recall, and F1 score under different thresholds.

Figure 6.3 further illustrates the threshold effect. As the threshold increases, the accuracy improves because the cases where normal records are wrongly classified as anomalies decrease. At the same time, the recall rate decreases because more attack records are missed due to being

below the threshold.

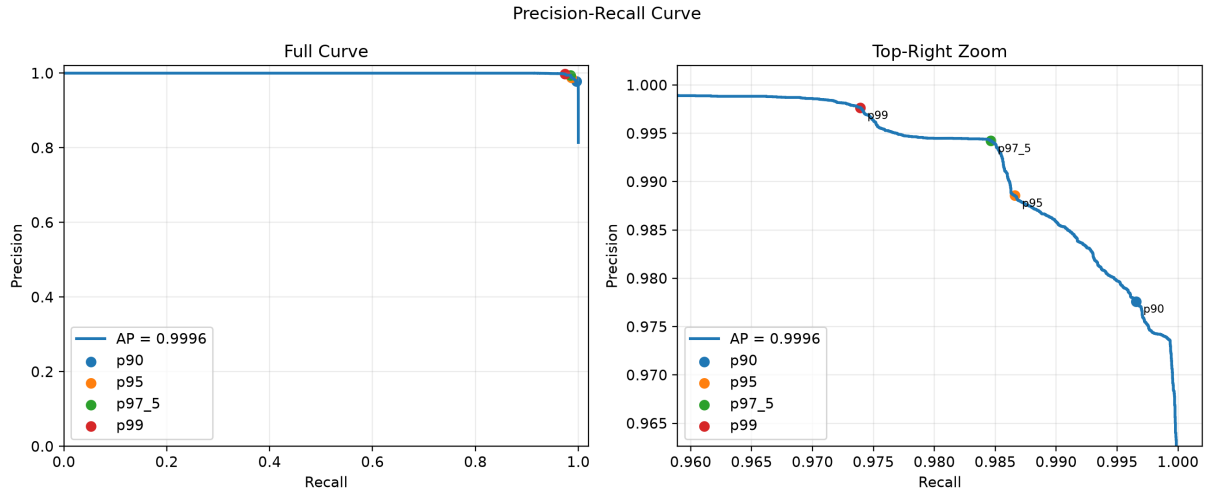


Figure 6.4: Precision-recall curve of the basic autoencoder.

The precision-recall curve shown in Figure 6.4 indicates that this model maintains high precision and recall rates across a wide range of thresholds. The average precision rate is 0.9996.

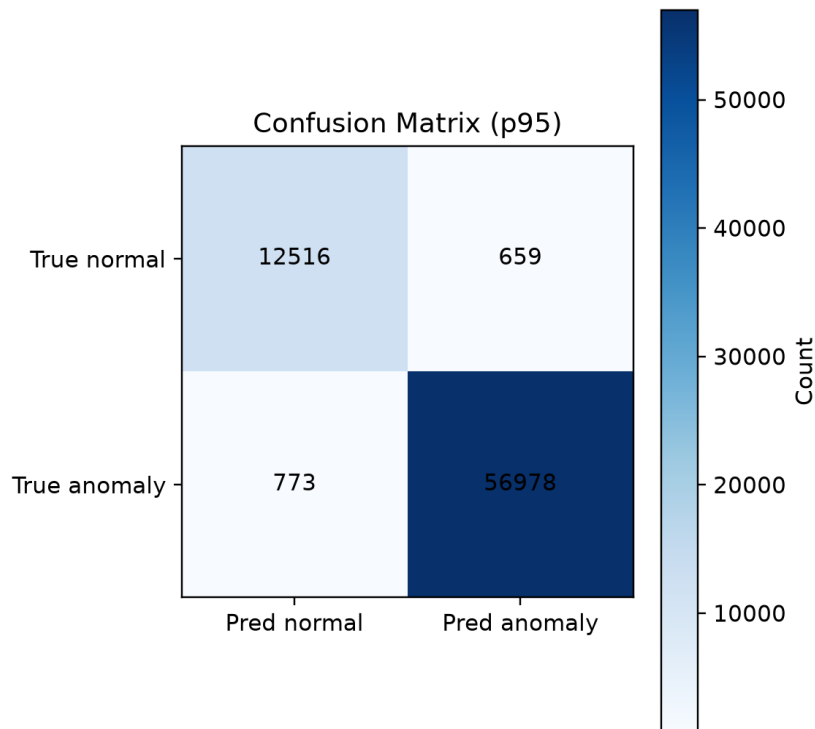


Figure 6.5: Confusion matrix of the basic autoencoder using the p95 threshold.

Figure 6.5 presents the confusion matrix under the main p95 threshold. This model correctly classified 12516 normal records and 56,978 abnormal records, generating 659 false positives and

773 false negatives. These results indicate that the basic autoencoder is effective in detecting most attack records, but not perfect. The reconstruction errors of some normal records are relatively high, while the reconstruction effects of some attack records are good and even below the abnormal threshold. Overall, the basic autoencoder provides a benchmark for this project. It learned a compact representation of normal network traffic and distinguished normal and abnormal records through reconstruction errors. The results also show that the selection of the threshold affects the balance between false alarms and missed attacks. Therefore, subsequent model improvements should not only focus on reducing reconstruction loss, but also strive to enhance the separation effect of normal and abnormal data in the reconstruction error distribution.

6.2 Results of Sparse Autoencoder (Hongyan)

6.2.1 Results of Sparse Autoencoder

This section reports the results of the sparse autoencoder using the final configuration selected by the hyperparameter studies, namely a sparsity weight $\beta = 10^{-3}$ and a sparsity target $\rho = 0.1$. Unless otherwise stated, all metrics use the primary p95 threshold.

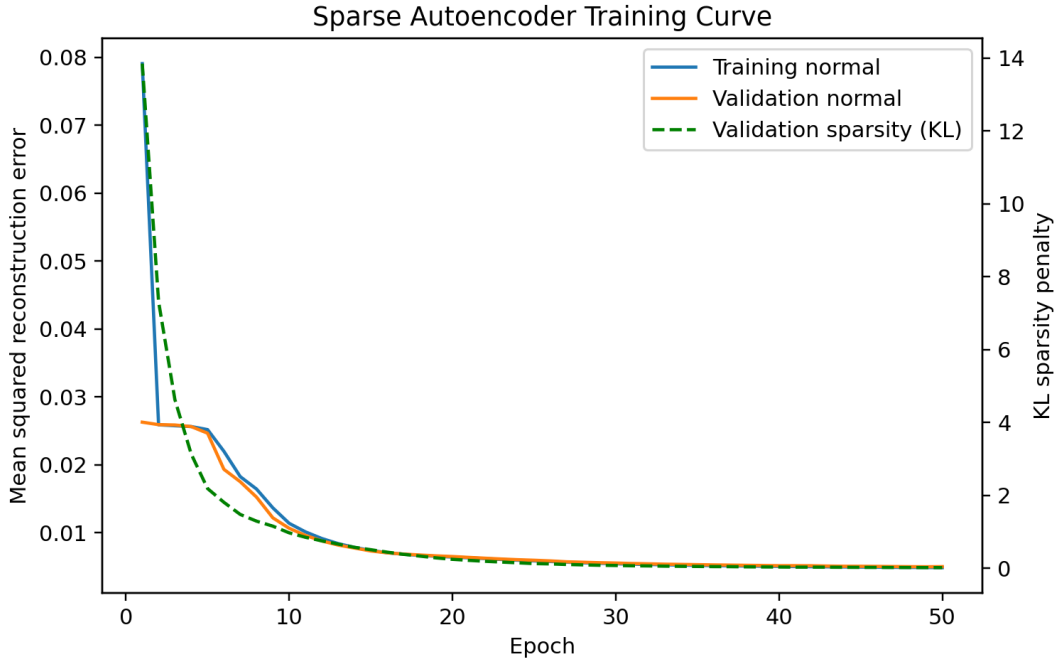


Figure 6.6: Training and validation reconstruction loss of the sparse autoencoder, with the validation sparsity penalty on the secondary axis.

Figure 6.6 shows the training and validation reconstruction loss, together with the validation sparsity penalty. Both reconstruction losses decrease quickly and stay close to each other, which indicates that the sparsity constraint does not cause overfitting. At the same time, the KL sparsity penalty decreases steadily during training, showing that the encoder gradually learns a sparse representation whose average activations approach the target ρ .

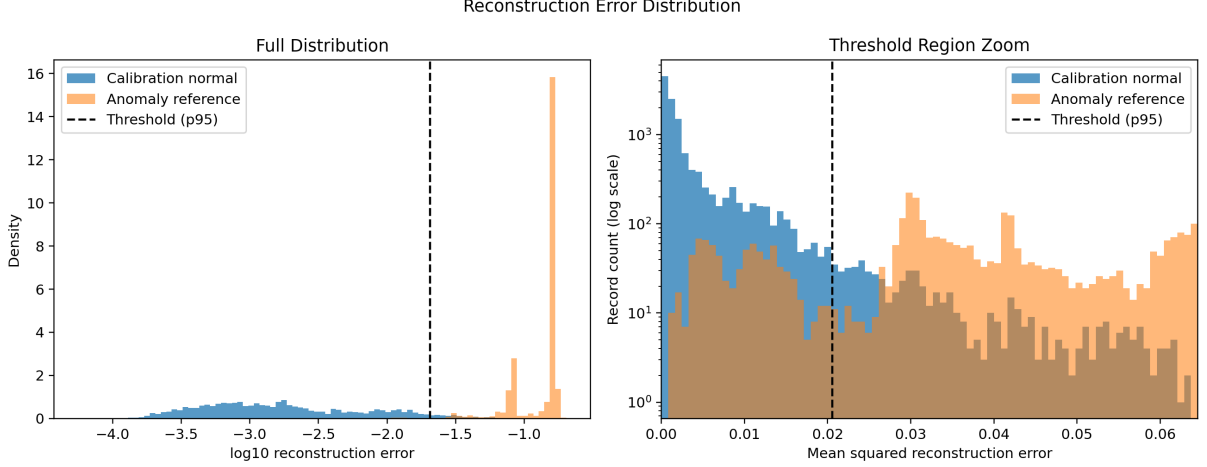


Figure 6.7: Reconstruction error distribution of the sparse autoencoder.

Figure 6.7 compares the reconstruction-error distributions of normal and abnormal records. The average reconstruction error is 0.005 for normal calibration records and 0.14 for abnormal records. This clear separation supports using the reconstruction error as the anomaly score. As with the basic model, a small overlap remains near the threshold region, which explains the residual false positives and false negatives.

Table 4: Sparse autoencoder performance under different percentile thresholds.

Threshold	Value	Precision	Recall	F1	Accuracy
p90	0.0134	0.9775	0.9895	0.9834	0.9728
p95	0.0205	0.9886	0.9864	0.9875	0.9797
p97.5	0.0303	0.9942	0.9774	0.9857	0.9769
p99	0.0447	0.9976	0.9556	0.9762	0.9620

Table 4 reports the performance under the four percentile thresholds. The pattern is consistent with the basic model: lower thresholds achieve higher recall at the cost of more false positives, while higher thresholds increase precision but miss more attacks. The p95 threshold provides a balanced operating point with precision 0.9886, recall 0.9864, and F1 0.9875.

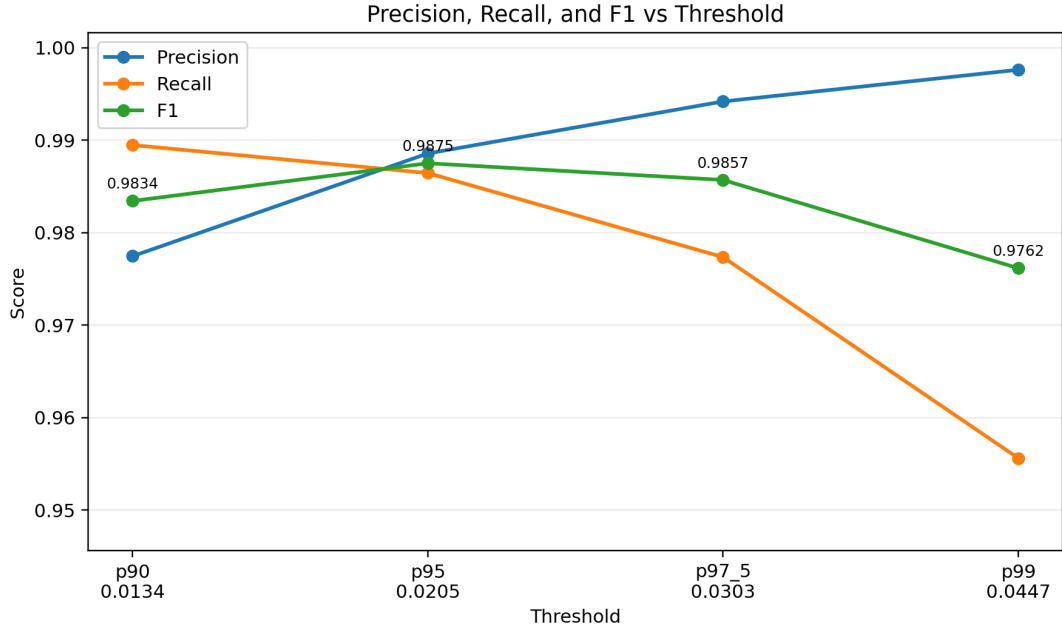


Figure 6.8: Precision, recall, and F1 score of the sparse autoencoder under different thresholds.

Figure 6.8 illustrates the threshold effect. As the threshold increases, the precision improves because fewer normal records are wrongly classified as anomalies, while the recall decreases because more attack records fall below the threshold and are missed.

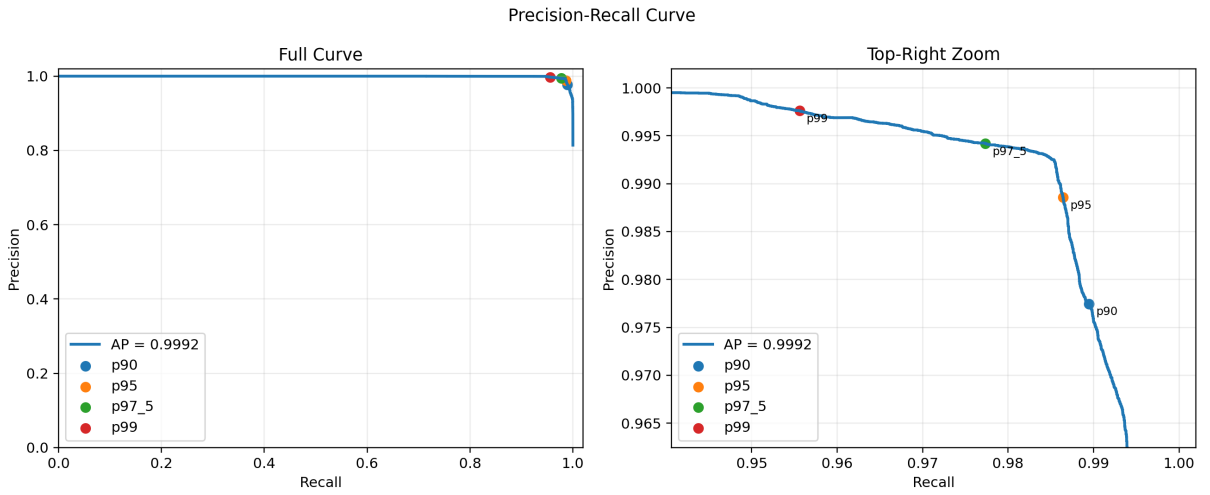


Figure 6.9: Precision-recall curve of the sparse autoencoder.

The precision-recall curve in Figure 6.9 shows that the sparse autoencoder maintains high precision and recall across a wide range of thresholds, which confirms that the reconstruction error is a strong anomaly score for this model.

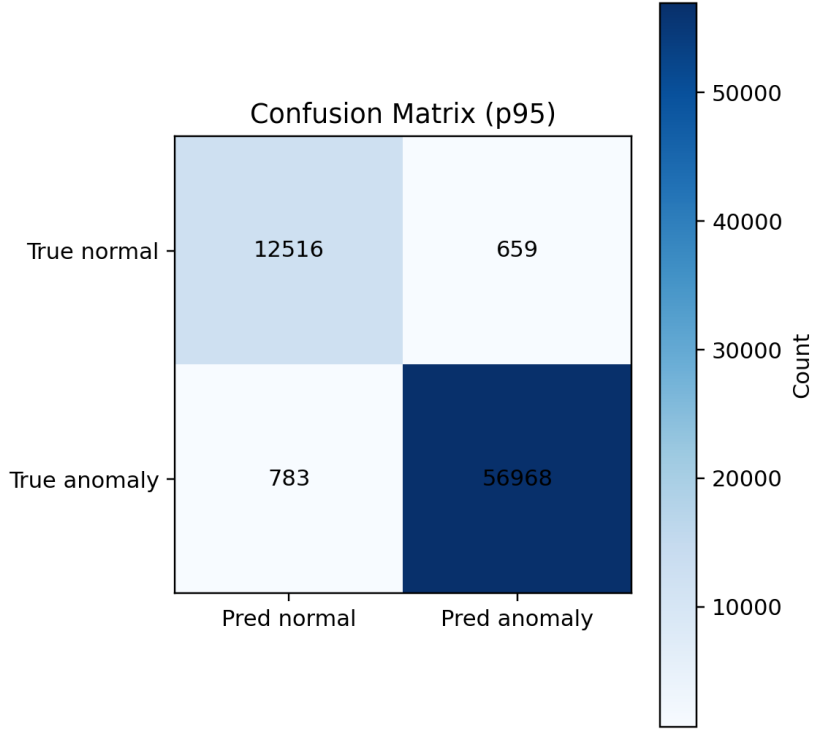


Figure 6.10: Confusion matrix of the sparse autoencoder using the p95 threshold.

Figure 6.10 presents the confusion matrix at the primary p95 threshold. The model correctly classifies 12,516 normal records and 56,968 abnormal records, producing 659 false positives and 783 false negatives.

6.2.2 Hyperparameter Study

Two additional studies were carried out to understand the effect of the sparsity hyperparameters. In each study, one hyperparameter was varied while all other settings were kept fixed, the p95 threshold was recalibrated for every configuration, and the model was evaluated on the anomaly reference set. In addition to the overall metrics, the recall on the two rare attack categories, R2L and U2R, is reported, because these categories are the most difficult to detect.

The first study varied the sparsity weight β with the sparsity target fixed at $\rho = 0.05$. The setting $\beta = 0$ corresponds to a non-sparse autoencoder of the same architecture and serves as a controlled reference. As reported in Table 5 and Figure 6.11, the precision and false positive rate stay almost constant across all settings, because the p95 threshold fixes the normal-traffic false positive rate by construction, so the differences appear only in recall. With this network capacity, the overall F1 score stays within a narrow band (0.9845–0.9877) for every value of

β , and even the non-sparse reference ($\beta = 0$) already reaches an F1 of 0.9873 with strong rare-attack recall (R2L 0.951, U2R 1.000). This shows that most of the detection ability comes from the autoencoder architecture itself, and that the sparsity penalty acts as a mild regularizer rather than a decisive factor. The main effect of β is on the recall of the R2L class, which is the smallest and hardest category and therefore the most sensitive: it drops to 0.593 at $\beta = 10^{-3}$ but stays high at the neighbouring settings. The best overall F1 is obtained at $\beta = 10^{-2}$ (0.9877), and $\beta = 10^{-3}$ was retained for the final model because, once combined with the sparsity target selected below, it gives the best trade-off between overall F1 and rare-attack recall.

Table 5: Effect of the sparsity weight β (sparsity target $\rho = 0.05$, p95 threshold).

β	Precision	Recall	F1	R2L recall	U2R recall
0	0.9886	0.9860	0.9873	0.951	1.000
10^{-4}	0.9885	0.9835	0.9860	0.940	0.942
10^{-3}	0.9885	0.9805	0.9845	0.593	0.981
10^{-2}	0.9886	0.9868	0.9877	0.949	0.981
10^{-1}	0.9886	0.9858	0.9872	0.972	0.981

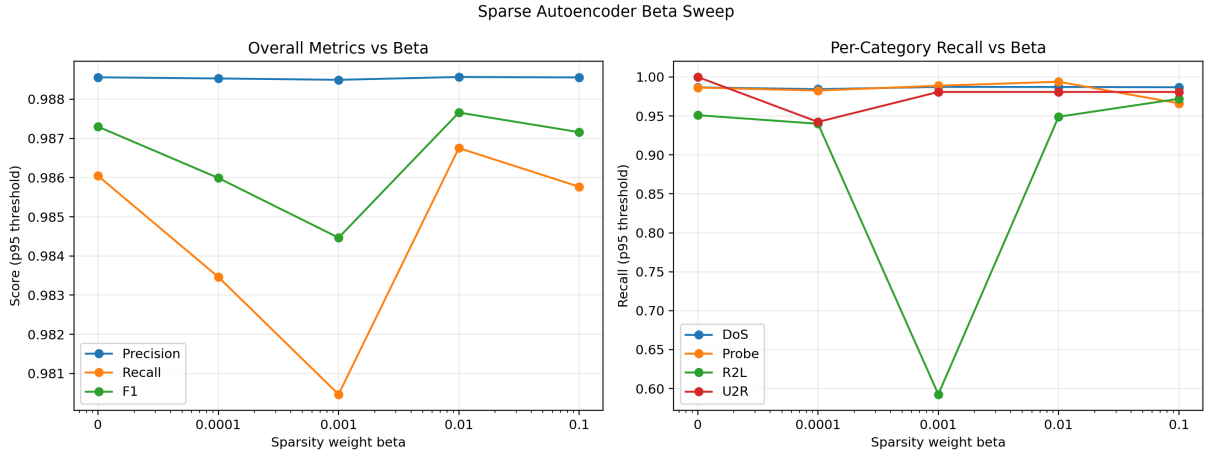


Figure 6.11: Overall metrics and per-category recall of the sparse autoencoder as a function of the sparsity weight β .

The second study fixed $\beta = 10^{-3}$ and varied the sparsity target ρ . As shown in Table 6 and Figure 6.12, the overall F1 score again stays within a narrow band (0.9825–0.9875), and the U2R recall is stable at 0.981 across all settings. As in the first study, the main source of variation is the recall of the R2L class, which is non-monotonic in ρ : it is high at $\rho = 0.1$ (0.952), drops sharply at $\rho = 0.05$ (0.593), and is lowest at the largest target $\rho = 0.2$ (0.474). This behaviour

is consistent with R2L being the smallest attack family whose records closely resemble normal traffic, so its detection is easily perturbed by small changes in the learned representation. The best operating point is $\rho = 0.1$, which achieves the highest F1 score (0.9875) and the strongest R2L recall among the tested targets, and it was therefore adopted together with $\beta = 10^{-3}$ for the final model.

Table 6: Effect of the sparsity target ρ (sparsity weight $\beta = 10^{-3}$, p95 threshold).

ρ	Precision	Recall	F1	R2L recall	U2R recall
0.01	0.9886	0.9857	0.9871	0.949	0.981
0.025	0.9886	0.9853	0.9869	0.868	0.981
0.05	0.9885	0.9805	0.9845	0.593	0.981
0.1	0.9886	0.9864	0.9875	0.952	0.981
0.2	0.9885	0.9767	0.9825	0.474	0.981

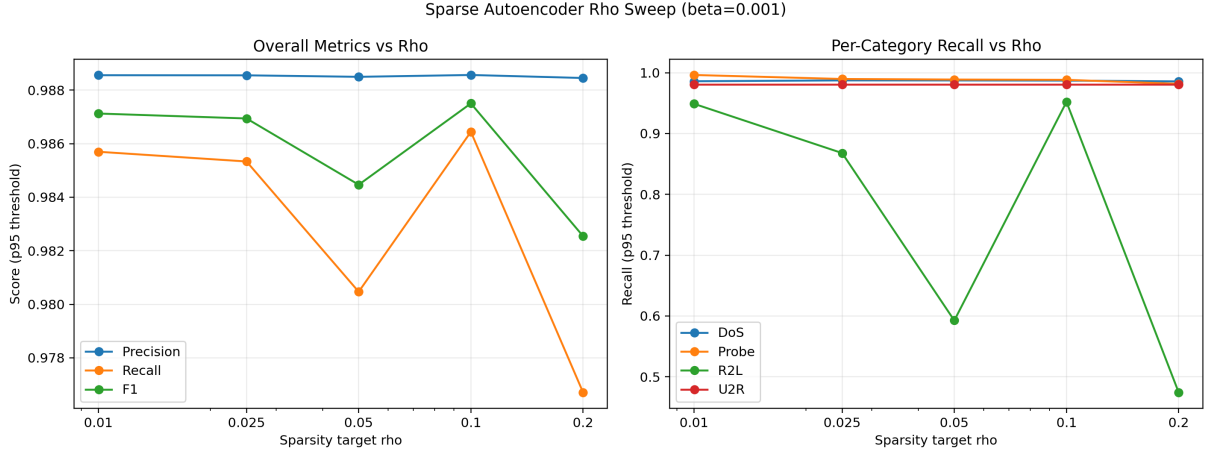


Figure 6.12: Overall metrics and per-category recall of the sparse autoencoder as a function of the sparsity target ρ .

Overall, these experiments show that, at this network capacity, the sparse autoencoder performs on par with the basic autoencoder: it achieves an F1 score of 0.9875 at the p95 threshold, with the same precision and false positive rate. The controlled comparison against the non-sparse reference ($\beta = 0$) indicates that most of the detection ability comes from the autoencoder architecture and the reconstruction-error scoring, while the sparsity constraint acts as a mild regularizer whose effect is small and configuration-dependent rather than decisive. This is a different conclusion from what is often reported for strongly bottlenecked autoencoders, and it suggests that the benefit of sparsity depends on model capacity: when the latent space is already expressive enough to reconstruct normal traffic well, the added sparsity penalty offers

limited additional separation. A remaining limitation is that the recall on the R2L class, which is the smallest attack family and most closely resembles normal traffic, is the most sensitive to the sparsity hyperparameters and remains the main target for future improvement.

6.3 Results of Deep Autoencoder (Anh)

Inheriting model’s hyperparameter from basic autoencoder, the results of the deep autoencoder model is present in this section. Similar to other models, all metrics use the chosen p95 threshold.

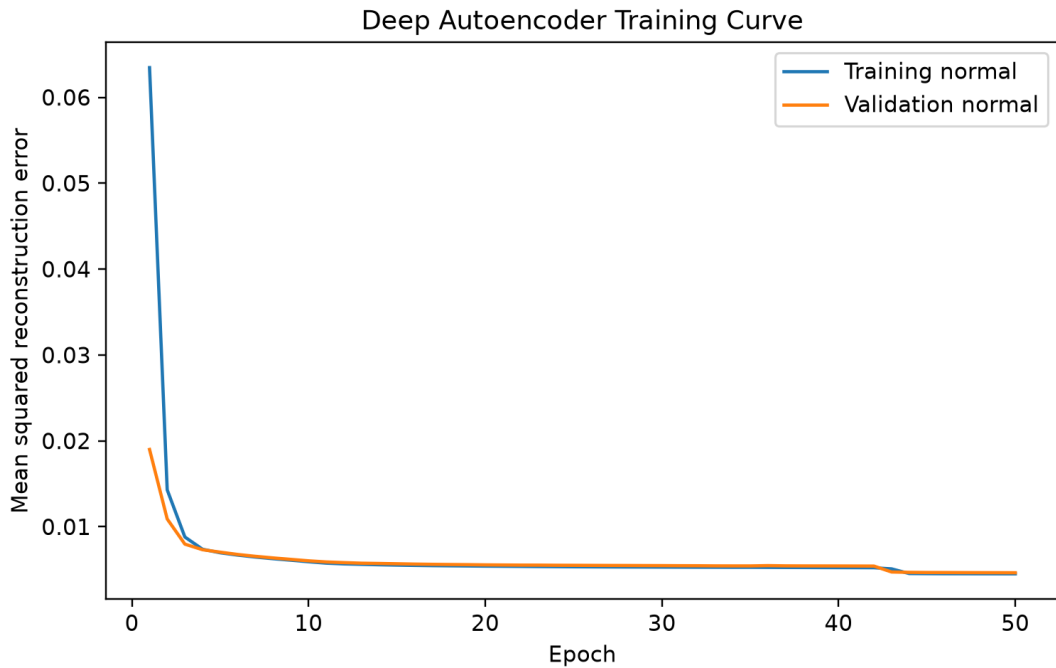


Figure 6.13: Training and validation reconstruction loss of the deep autoencoder.

As shown in Figure 6.13, both the training and validation loss on normal data decrease sharply in the first few epochs and continue to decrease gradually afterward, indicating that the model is learning the underlying structure of normal traffic. The validation loss plateaus around epoch 10, drops again near epoch 30, and stabilizes at approximately 0.0046 by epoch 50.

The reconstruction error was displayed in Figure 6.14. According to the evaluation report, the re-build error is about 0.005 for normal data and about 0.14 for attack data. This large gap suggests that the model generates highly different reconstruction errors for normal and abnormal data, implying that it may be used as an effective anomaly score for classification.

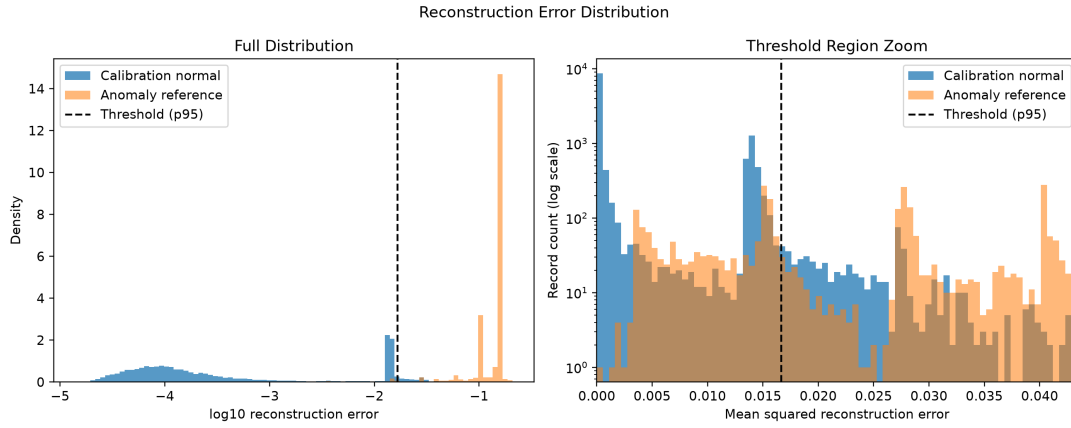


Figure 6.14: Reconstruction error of the deep autoencoder.

Table 7 presents the performance of the deep autoencoder under various percentile thresholds and the matching threshold values that are obtained from the error distribution. The model follows the same trend as the basic and sparse autoencoder models, with lower thresholds favoring recall at the expense of false positives and higher thresholds favoring accuracy at the expense of missed attacks.

Table 7: Deep autoencoder performance under different percentile thresholds.

Threshold	Value	Precision	Recall	F1	Accuracy
p90	0.0144	0.9774	0.9864	0.9819	0.9703
p95	0.0166	0.9885	0.9767	0.9825	0.9717
p97.5	0.0243	0.9942	0.9744	0.9842	0.9745
p99	0.0305	0.9976	0.9630	0.9800	0.9680

The similar threshold effect seen in the basic model is shown in Figure 6.15: recall declines as more attack recordings fall below the threshold and are missed, while accuracy improves as the threshold rises because fewer normal records are mistakenly categorized as anomalies.

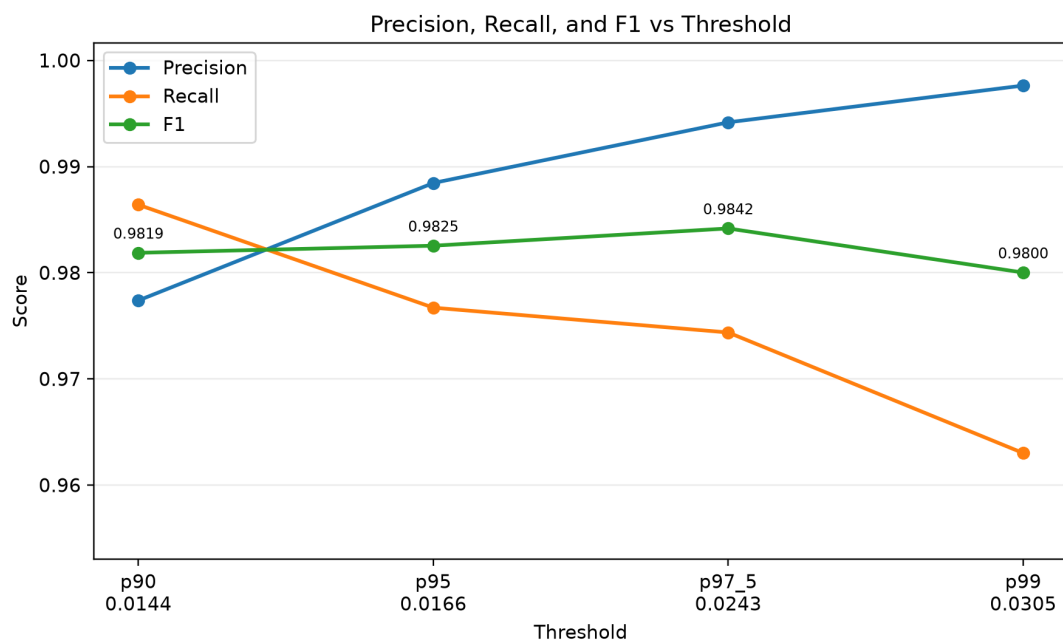


Figure 6.15: Precision, recall, and F1 score under diffe of the deep autoencoder.

With an average precision of 0.9990, the precision-recall curve in Figure 6.16 demonstrates that the model retains good precision and recall across a wide range of thresholds.

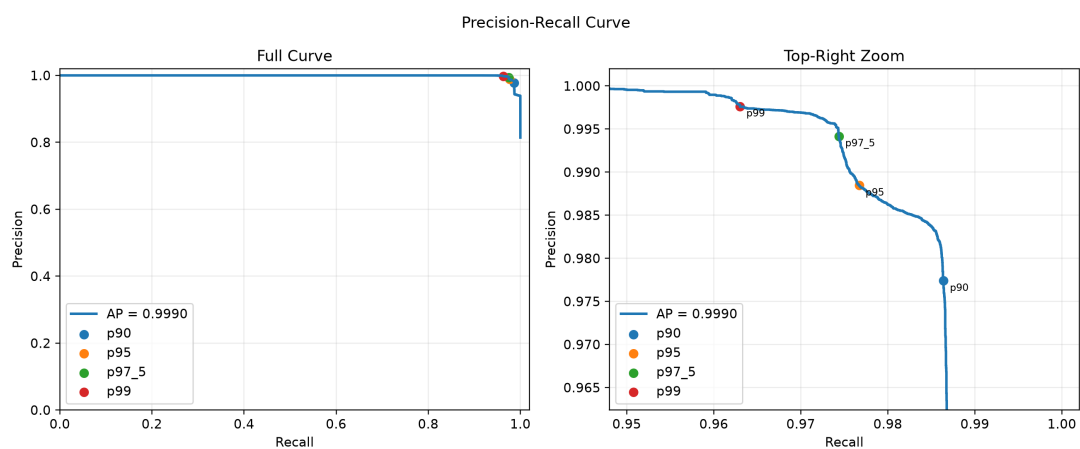


Figure 6.16: Precision-recal of the deep autoencoder.

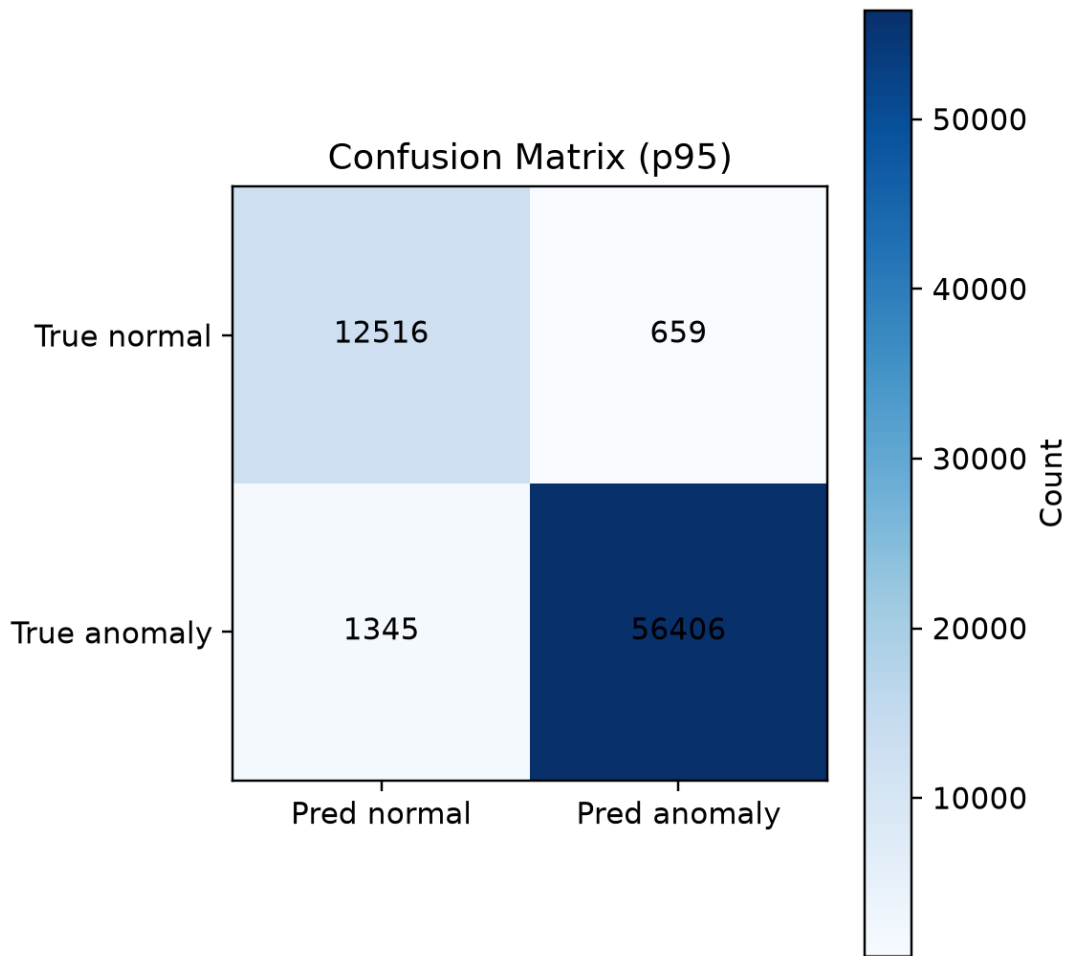


Figure 6.17: Confusion matrix of the deep autoencoder using the p95 threshold.

The confusion matrix for the deep autoencoder at the primary p95 threshold is illustrated in Figure 6.17. The model accurately identified 12,516 in normal calibration records as normal, which is the same number as the basic and sparse models. However, out of 57,751 anomaly reference data, 1,345 attacks were overlooked and classed as normal, which is much greater than the other two models. This is driven primarily by the R2L category, where recall drops to only 51%. This shows that greater depth does not guarantee better separation between normal and anomalous traffic. Without additional techniques, deeper networks are harder to train, as the gradient must travel back through more layers and can shrink or distort along the way, leading to poor or unstable weight updates during training. A more detailed comparison is presented in Section 6.4.

6.4 Model Comparison (Manisurya & Anh)

The performance of the Basic, Deep, and Sparse Autoencoder models was compared using Precision, Recall, F1 Score, and Accuracy. These evaluation metrics provide a comprehensive assessment of each model’s anomaly detection capability and allow us to identify the most effective architecture.

6.4.1 Performance Comparison

Table 8 summarizes the evaluation results of the Basic, Deep, and Sparse Autoencoder models. Precision, Recall, F1 Score, and Accuracy were used to compare the anomaly detection performance of each model. The results show that all three Autoencoder models achieved high

Table 8: Performance Comparison of Autoencoder Models

Model	Precision	Recall	F1 Score	Accuracy
Basic Autoencoder	0.988566	0.986615	0.987590	0.979810
Deep Autoencoder	0.988452	0.976710	0.982546	0.971745
Sparse Autoencoder	0.988564	0.986442	0.987502	0.979669

performance in detecting network anomalies. The Basic Autoencoder obtained the highest Accuracy (97.98%) and F1 Score (98.76%), indicating the best overall performance. The Sparse Autoencoder achieved very similar results, while the Deep Autoencoder showed slightly lower Recall and Accuracy. The lower result in Deep Autoencoder demonstrates that increasing network depth did not enhance anomaly identification. This shows that the additional hidden layers increased model complexity while not enhancing the representation of regular traffic with the existing training data. Overall, all models performed well, but the Basic Autoencoder provided the most balanced and accurate performance on the evaluation dataset.

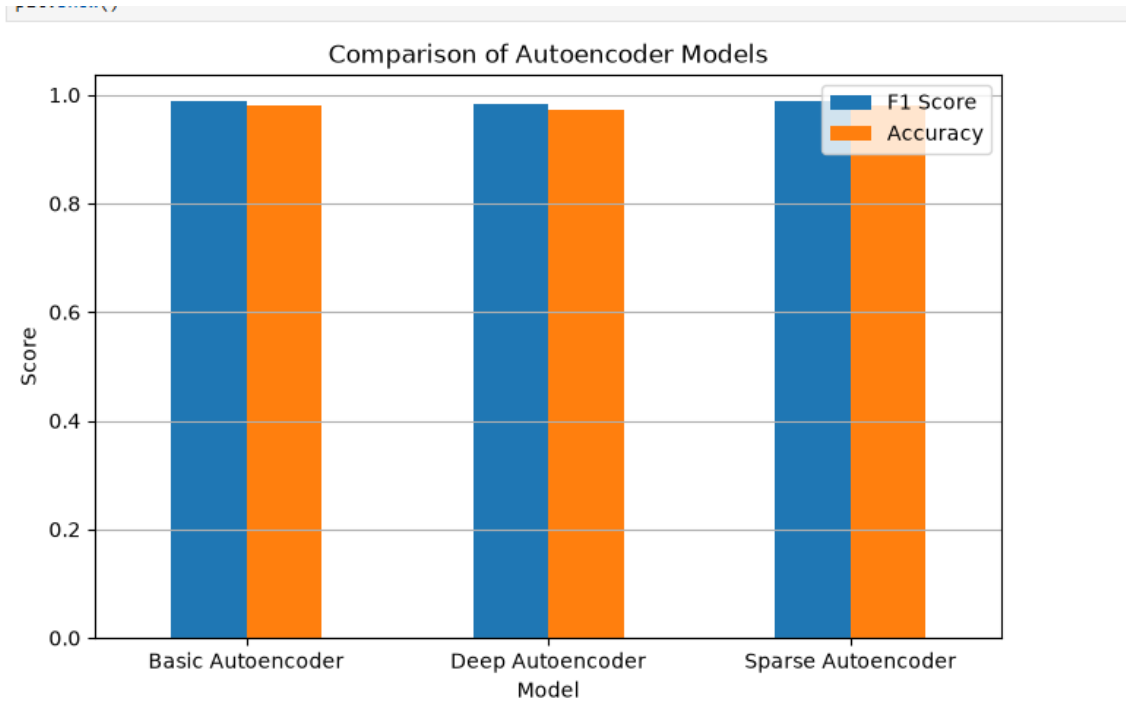


Figure 6.18: Comparison of Autoencoder Models using F1 Score and Accuracy.

Figure 6.18 compares the F1 Score and Accuracy of the Basic, Deep, and Sparse Autoencoder models. The Basic Autoencoder achieved the highest overall performance with the highest F1 Score and Accuracy. The Sparse Autoencoder produced very similar results, while the Deep Autoencoder showed a slightly lower Accuracy and F1 Score. Overall, all three models demonstrated strong anomaly detection capability, with only minor differences in performance.

Table 9: Per-Attack Recall Comparison of Autoencoder Models

Model	DoS	Probe	R2L	U2R
Basic Autoencoder	0.999	0.982	0.951	1.000
Sparse Autoencoder	0.999	0.982	0.952	0.981
Deep Autoencoder	0.999	0.981	0.510	0.981

As mentioned in Section 6.3, the confusion metric shows that a significantly larger number of anomaly records are classified as normal. This performance decline is due to the poor R2L recall of just 0.510. Unlike DoS or Probe attacks, R2L assaults carry out illegal activity while retaining traffic patterns that closely mimic genuine network connections. It is more challenging to differentiate them based only on reconstruction error since their connection characteristics, such as length, byte counts, and TCP flag sequences, frequently overlap with typical traffic. The Deep Autoencoder’s extra hidden layers expand the model’s representational capacity, allowing it to reconstruct not just regular traffic but also nuanced attack patterns with relatively low

reconstruction error. Thus, many R2L assaults fall below the abnormality threshold and are mistakenly categorized as normal. This suggests that increasing network depth alone may not necessarily improve anomaly detection performance and can even reduce sensitivity to stealthy attacks, unless additional regularization techniques are incorporated, such as residual connections, batch normalization, or dropout.

7 Recommendations for Future Work (Ziqi & Anh)

Future work can improve the anomaly detection system by exploring more advanced Autoencoder architectures such as Variational Autoencoders (VAEs) or Transformer-based Autoencoders. VAEs could provide probabilistic anomaly scores, while recurrent architectures (LSTM or GRU) and Transformer-based autoencoders could be evaluated to capture temporal dependencies in network traffic. Additional hyperparameter tuning, including learning rate, latent dimension, and batch size, may further improve performance. Testing the models on larger and more diverse cybersecurity datasets such as CIC-IDS2017 [11] and UNSW-NB15 [12], would provide a stronger assessment of robustness and generalization. Finally, deploying the trained model in a real-time intrusion detection system would enable continuous monitoring and faster anomaly detection.

References

- [1] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *Proceedings of the MLSDA Workshop*, 2014, pp. 4–11.
- [2] K. Alrawashdeh and C. Purdy, “Toward online anomaly detection in network traffic using deep learning,” in *IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2016, pp. 194–200.
- [3] S. J. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. K. Chan, “Cost-based modeling for fraud and intrusion detection,” in *DARPA Information Survival Conference and Exposition*, vol. 1, 2000, pp. 130–144.

- [4] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” *arXiv preprint arXiv:1802.07389*, 2018.
- [5] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2019, vol. 32.
- [6] A. Ng, “Sparse autoencoder,” in *CS294A Lecture Notes*, 2011.
- [7] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, “Tensorflow: A system for large-scale machine learning,” *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 265–283, 2016.
- [8] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2014. [Online]. Available: <https://doi.org/10.48550/arXiv.1412.6980>
- [9] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [10] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith *et al.*, “Array programming with numpy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [11] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.
- [12] N. Moustafa and J. Slay, “Unsw-nb15: A comprehensive data set for network intrusion detection systems,” in *Military Communications and Information Systems Conference (MilCIS)*. IEEE, 2015.